IFCD107 · ESPECIALISTA EN INTELIGENCIA ARTIFICIAL

Demostrador y documentación

Cómo entregar un proyecto de IA para que **otra persona pueda ejecutarlo**

Sprint 16 · S16_01 · 20 min de teoría el 4/12 · cuaderno: 45 min en clase (se ejecuta y se lee) y material de consulta

LA IDEA · QUÉ SE ENTREGA Y CUÁNDO

Entregar es que otra persona lo ejecute

Un modelo que solo vive en tu portátil no es un producto.

Cuándo	Qué tiene que estar hecho
Viernes 4/12	**Demostrador funcionando** (aunque sea básico) y esqueleto de la memoria
Lunes 7 y martes 8/12	**Sin clase**: buen momento para avanzar la memoria en equipo
Miércoles 9/12	Sesgos mitigados, demostrador y model card casi listos; memoria con todos los apartados
Jueves 10/12, 14:30	**Congelación** (desde ahí no se cambia nada): último push. Lo que no esté subido, no cuenta

Se sube a **sprint-16/**

9 ficheros del equipo y **1 hoja por persona**: README, evaluación, demostrador, riesgos, model card y memoria.

Aparte: el vídeo de reserva va en **docs/** y la coevaluación, al docente.

Palabras nuevas

Demostrador: app o cuaderno donde otra persona prueba el modelo.

Model card: la ficha del modelo (qué hace, con qué datos, cuándo no usarlo).

LA IDEA · EL DEMOSTRADOR

El demostrador no reentrena

1 · Se entrena una vez
En el cuaderno o en un script de entrenamiento

→

2 · Se guarda
El modelo con **joblib** y un archivo de **metadatos.json**

→

3 · Se carga
cargar_modelo() lee los dos archivos

→

4 · Se usa
La app, los widgets o la API piden una predicción

LA CARPETA

```
demostrador/
├── modelo/              # joblib, json
├── logica.py            # validar + predecir
├── app.py               # interfaz Streamlit
├── api.py               # (opcional) FastAPI
├── tests/               # pruebas
├── requirements.txt     # versiones fijas
└── README_demostrador.md
```

METADATOS.JSON (EXTRACTO REAL)

```
{
  "campos": {"noches": {"tipo": "entero", "min": 1, "max": 30, ...}, ...},
  "auc_test": 0.746,
  "n_entrenamiento": 4800,
  "semilla": 42,
  "versiones": {"scikit-learn": "1.9.1", ...},
  "limitaciones": "Entrenado con datos sintéticos..."
}
```

LA IDEA · LA LÓGICA SEPARADA DE LA INTERFAZ

La lógica, separada de la interfaz

INTERFACES: CAPAS FINAS

- **app.py** Streamlit
- **widgets** en el cuaderno
- **api.py** FastAPI, opcional
- **tests/** pytest

→

logica.py

```
cargar_modelo()
validar_reserva()
predecir_reserva()
```

Sin nada de interfaz

→

modelo/

```
modelo_cancelaciones.joblib
metadatos.json
```

1 · Errores en español y accionables

Dicen **qué campo falla**, qué se esperaba y qué se escribió. Salida real del cuaderno:

'noches' debe estar entre 1 y 30 (has escrito -3).

2 · Aviso de incertidumbre en cada respuesta

Viaja **con** el resultado: incluye la nota del modelo (AUC de 0,746 en datos de prueba), que se equivoca con frecuencia, y sus límites.

3 · Devuelve datos, no imprime

Un diccionario que cada interfaz muestra como quiera:

probabilidad 0,812 · **nivel** alto · **tasa_base** 0,459 · **aviso** …

Si hay errores, solo devuelve **errores**.

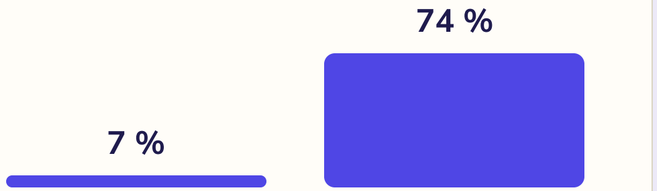
LA IDEA · PRUEBAS DE SENTIDO COMÚN

Un modelo puede fallar sin dar error

EJEMPLO INVENTADO: ¿SE DEVOLVERÁ TARDE UN PRÉSTAMO?

```
def test_sentido_comun():
    largo = predecir_prestamo({**base, "dias": 42, "novedad": 1})
    corto = predecir_prestamo({**base, "dias": 7, "novedad": 0})
    assert largo["probabilidad"] > corto["probabilidad"]
```

Modelo bien entrenado: **pytest -q** muestra **.**



Etiquetas al revés: **pytest -q** muestra **F**



Además, a mano: cinco entradas

3 normales: resultado coherente.

1 imposible (por ejemplo, 250 años): mensaje claro, sin romperse.

1 vacía: mensaje claro.

LA IDEA · OPCIONES DE DEMOSTRADOR

Cuatro formas de enseñar el modelo

Opción	Cuándo	Ventaja	Inconveniente	En el cuaderno
Streamlit (recomendada)	Modelos tabulares, series, texto	Web interactiva con pocas líneas	Hay que instalarlo (**streamlit run**)	Sección 4
Gradio	Imágenes, audio, chat	Muy rápido para subir una foto	Menos control del diseño	Sin sección
Cuaderno con widgets	Plan B, sin instalar nada	Funciona en Colab	Menos «producto»	Sección 5
API con FastAPI (opcional)	Si otro sistema lo consulta	Estándar en producción	Necesita un cliente que la use	Sección 6

app.py es una capa fina

1. Carga el modelo una vez (**cargar_modelo**, en caché).
2. Dibuja el formulario con **meta["campos"]**.
3. Llama a **predecir_reserva**.
4. Muestra el resultado o los errores.

En Colab, Streamlit no se abre

Hace falta un túnel. Para el plan B usad la opción de **widgets**, que funciona dentro del cuaderno.

En clase solo se revisa que **app.py** compila; se lanza desde una terminal.

LA IDEA · REPRODUCIBILIDAD Y CONGELACIÓN

Reproducible y congelado a las 14:30

Lista de reproducibilidad del demostrador (cuaderno, sección 7)

- ✓ **requirements.txt** con **versiones fijas** (**pip freeze**, recortado a lo que usas)
- ✓ El **modelo se crea** con un script o cuaderno y el README dice cómo
- ✓ El README dice **qué comandos ejecutar** y qué se verá
- ✓ La app **no se rompe** con entradas erróneas y avisa de la incertidumbre
- ✓ Hay **pruebas** (**pytest -q**) y un **vídeo de reserva** de 90 s
- ✓ **Ni claves ni datos personales** en el repositorio

14:30

10/12 · congelación

Último push. Antes, la etiqueta **v1.0-entrega**.

Después no se cambia ni código ni memoria (sí la presentación y el vídeo).

Prueba en limpio

Otra persona, que **no escribió el README**, lo sigue en un Colab nuevo (Code → Download ZIP) y anota lo que falla.

Push = subir los cambios al repositorio · **etiqueta (tag)** = marca de Git que señala la versión entregada · **congelación** = desde esa hora no se cambia nada. Se pasan los seis bloques de **checklist_congelacion.md**.

LA IDEA · MINI-RETO DEL CUADERNO (CASA)

Mini-reto: adapta la plantilla a tu proyecto

1 · Modelo y metadatos
Guarda tu mejor modelo

→

2 · logica.py
Campos, validaciones y aviso propios

→

3 · app.py
Lánzalo con **streamlit run**

→

4 · Tres pruebas
De sentido común, con **pytest**

→

5 · Vídeo de 90 s
Plan B de la demo

Mini-reto: adapta la plantilla a tu proyecto. Las **preguntas de reflexión** del cuaderno se comentan con el docente.

PRÁCTICA · CUADERNO S16_01

Lo que haremos en el cuaderno

Parte	Qué haremos	Dónde
Preparación	Preparar las herramientas	Clase
Ejemplos	Entrenar el modelo y guardar los metadatos	Clase
Ejemplo	Escribir **logica.py** y probarla	Clase
Ejercicio 1	Comprobar la coherencia entre campos	Casa
Ejercicio 2	Convertir la lista de errores en un mensaje	Casa
Ejercicio 3	Dos reservas de comparación	Casa
Ejemplo	Pruebas de sentido común (**tests/test_logica.py**)	Clase
Ejemplo	Generar **app.py** con Streamlit	Clase
Secciones 5 a 7	Widgets, API opcional y **requirements.txt**	Casa
Cierre	Mini-reto y preguntas de reflexión	Casa

Cada ejercicio: **lee** → **escribe** → **ejecuta** → **comprueba el** ✔. La solución está plegada: ábrela tras intentarlo.

CONSULTA

Material de consulta: código y aclaraciones

Cuándo abrirlo

Después de intentar el ejercicio, o cuando algo no te salga. En clase no se proyecta.

Cómo está ordenado

En el **mismo orden que el cuaderno**, y después las del reto: sesgos, model card y memoria.

Parte	Tema	Diapositivas
Sección 2	logica.py: predecir y avisar	11
1	Coherencia entre campos	12 y 13
2	Mensaje de errores	14 y 15
3	Dos reservas de comparación	16 y 17
Sección 3	Pruebas de sentido común	18
Sección 6	API: protección básica	19
Reto	Sesgos, model card y memoria	20, 21 y 22

CONSULTA · SECCIÓN 2 · LOGICA.PY

Escribir logica.py y probarla

```
reserva_ejemplo = {"antelacion_dias": 120, "noches": 5, "adultos": 2, "ninos": 1, "precio_noche": 95.0,
    "cliente_repetidor": 0,
                   "cancelaciones_previas": 0, "tipo_habitacion": "estandar", "canal": "web_intermediaria",
                       "tarifa": "flexible", "deposito": "sin_deposito"}
print(logica.predecir_reserva(reserva_ejemplo, modelo_cargado, meta_cargado))
print(logica.predecir_reserva({**reserva_ejemplo, "noches": -3, "canal": "telepatia"}, modelo_cargado, meta_cargado))
```

0,812

Probabilidad de cancelación

alto · 0,459

Nivel · media histórica

Con **noches = -3** y un canal inventado solo devuelve **errores**, sin probabilidad.

CONSULTA · EJERCICIO 1

Comprobar la coherencia entre campos

```
def validar_coherencia(reserva):
    errores = []
    if reserva["ninos"] > 0 and reserva["adultos"] < 1:
        errores.append("Los menores deben viajar con al menos un adulto.")
    if reserva["noches"] * reserva["precio_noche"] > 5000:
        errores.append("El importe total supera los 5000 €: revisa las noches y el precio.")
    return errores
err = validar_coherencia({**reserva_ejemplo, "noches": 20, "precio_noche": 400.0})
```

✔ correcto

Salida del cuaderno

8.000 €

Importe de la prueba

Dos **if independientes**: pueden salir los dos errores; si todo es coherente, devuelve una **lista vacía**.

CONSULTA · ACLARACIONES · EJERCICIO 1

Leer la coherencia (ejercicio 1)

Dos **if** seguidos

Cada regla es **independiente**: pueden salir **los dos errores** a la vez.

return errores

Devuelve una **lista**, **vacía** si la reserva es coherente.

Resultado

Con 20 noches a 400 € (**8.000 €**) sale **1 error**: «El importe total supera los 5000 €…».

Errores típicos

Condición al revés (siempre da error) o un texto distinto al esperado.

CONSULTA · EJERCICIO 2

Convertir la lista de errores en un mensaje

```
def mensaje_errores(errores):
    if not errores:
        return ""
    return "No se puede calcular todavía:\n" + "\n".join(f"- {e}" for e in errores)
print(mensaje_errores(logica.validar_reserva({**reserva_ejemplo, "noches": -3, "canal": "telepatia"}, meta_cargado)))
```

2

Líneas de error en la prueba

una cadena vacía

Sin errores devuelve

Cabecera «No se puede calcular todavía:» y una línea **«- error»** por cada uno.

CONSULTA · ACLARACIONES · EJERCICIO 2

Leer el mensaje de errores (ejercicio 2)

if not errores: return ""

Sin errores, una cadena **vacía**: no hay nada que avisar.

"\n".join(f"- {e}" for e in errores)

Una línea por error, empezando por **-** , unidas con saltos de línea.

Resultado

La cabecera y **2 líneas**: noches fuera de rango y canal inventado.

Errores típicos

Devolver la cabecera aunque no haya errores o **olvidar el -** delante de cada error.

CONSULTA · EJERCICIO 3

Dos reservas de comparación

```
reserva_riesgo_alto = {**reserva_ejemplo, "antelacion_dias": 250, "deposito": "sin_deposito",
    "tarifa": "flexible", "canal": "web_intermediaria"}
reserva_riesgo_bajo = {**reserva_ejemplo, "antelacion_dias": 7, "deposito": "total", "tarifa": "no_reembolsable",
    "canal": "directo"}
p_alto = logica.predecir_reserva(reserva_riesgo_alto, modelo_cargado, meta_cargado)["probabilidad"]
p_bajo = logica.predecir_reserva(reserva_riesgo_bajo, modelo_cargado, meta_cargado)["probabilidad"]
```

95 %
Riesgo alto

4 %
Riesgo bajo

Diferencia **mayor de 0,30**: el modelo tiene sentido común; si puntuara igual a las dos, fallaría sin dar error.

CONSULTA · ACLARACIONES · EJERCICIO 3

Leer las dos reservas (ejercicio 3)

{**reserva_ejemplo, "campo": valor}

Copia un caso **válido** y cambia **solo** lo necesario.

["probabilidad"]

predecir_reserva devuelve un diccionario: se toma la **clave probabilidad**.

Resultado

Riesgo alto **95 %** y riesgo bajo **4 %**: la diferencia supera **0,30**.

Errores típicos

Un valor que no existe (p. ej. **"flexibe"**) o dejar un **campo sin escribir**.

CONSULTA · SECCIÓN 3 · PRUEBAS DE SENTIDO COMÚN

Las pruebas de tests/test_logica.py

```
def test_reserva_valida_devuelve_probabilidad():
    r = logica.predecir_reserva(reserva_ejemplo, modelo_cargado, meta_cargado)
    assert 0 <= r["probabilidad"] <= 1 and r["nivel"] in ("bajo", "medio", "alto") and "aviso" in r
def test_entrada_invalida_no_rompe():
    r = logica.predecir_reserva({**reserva_ejemplo, "noches": -3}, modelo_cargado, meta_cargado)
    assert r["errores"] and "probabilidad" not in r
def test_falta_un_campo():
    incompleta = {k: v for k, v in reserva_ejemplo.items() if k != "canal"}
    assert any("canal" in e for e in logica.validar_reserva(incompleta, meta_cargado))
def test_sentido_comun():
    assert p_alto > p_bajo
```

Salida: **OK** en las cuatro pruebas. Se guardarían en **tests/test_logica.py** y se lanzan con **pytest -q**.

CONSULTA · SECCIÓN 6 · API CON FASTAPI (OPCIONAL)

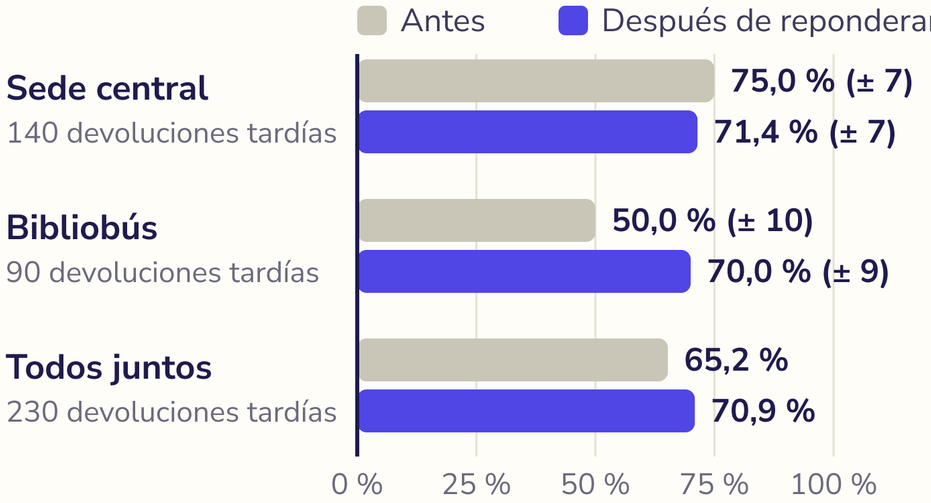
Una API no se expone sin protección

Riesgo	Medida	Cómo se nota
Cualquiera la usa	Clave de API o token en cada petición	Sin clave, error 401
Datos que viajan sin cifrar	HTTPS, nunca HTTP	Candado en el navegador
Entradas raras o mal formadas	Validar la entrada (en FastAPI, con **pydantic**)	Lo que no encaja, error 422
Abuso o saturación	Límite de peticiones por clave	Evita caídas y costes
Claves en el código	Variables de entorno, nunca en el repositorio	Ninguna clave en Git

Aunque la API sea opcional, cuenta en la memoria qué protección básica tiene.

CONSULTA · RETO · TUTORÍA DE SESGOS (EJEMPLO INVENTADO)

La media puede esconder un sesgo



Exhaustividad (recall): de las devoluciones tardías, ¿cuántas detecta el modelo? Entre paréntesis, la duda aproximada del 95 % en puntos.

Mirar por subgrupo

Con todos juntos, el modelo detecta el 65,2 % de las tardías. Pero en el bibliobús solo el 50,0 %.

Con solo 90 casos, hay una duda de unos ± 10 puntos; aun así, la brecha de 25 puntos es mayor.

Medir lo que cuesta

Más peso al bibliobús al entrenar (**reponderar**) lo sube a 70,0 %; la sede central baja 3,6 puntos.

A cambio, hay más falsas alarmas: precisión del bibliobús, de 64,3 % a 58,3 %.

Aplicar una mitigación no basta: **medid antes y después, por subgrupo**.

CONSULTA · RETO · MODEL CARD · 85 MIN

La model card: la etiqueta del modelo

1 · Detalles del modelo
Nombre, versión, librería con su versión y semilla

2 · Uso previsto
Para qué, para quién, qué decisión sigue siendo humana y qué usos NO se permiten

3 · Datos
Fuente, licencia, tamaño, cómo se partió y sesgos de muestreo

4 · Factores relevantes
Grupos o condiciones que cambian el rendimiento

5 · Métricas
Baseline frente al modelo, con su incertidumbre

6 · Rendimiento por subgrupos
Una tabla: n, métrica y diferencia con el global

7 · Análisis ético y de riesgos
AI Act y RGPD, mitigaciones y supervisión humana

8 · Limitaciones y recomendaciones
Cuándo no fiarse y qué haría falta antes de un despliegue real

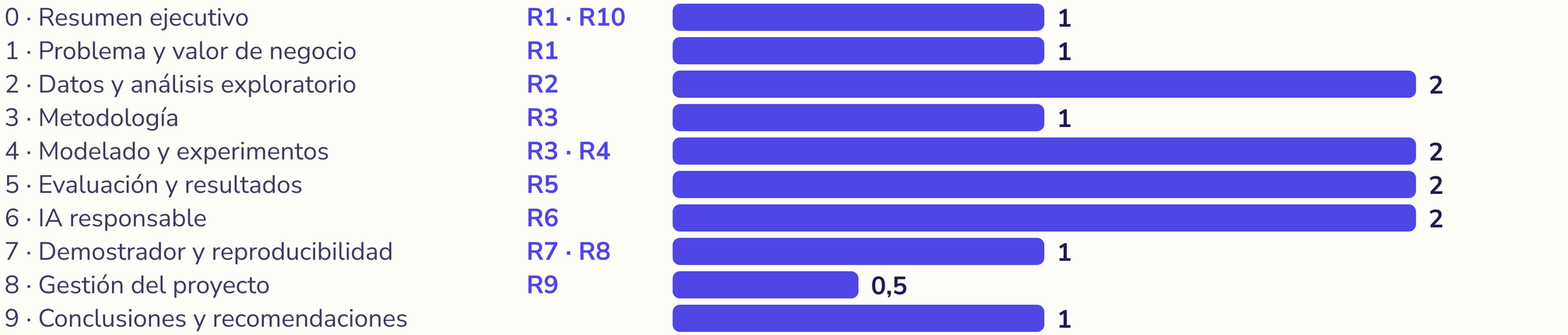
9 · Contacto y mantenimiento
Quién responde y cada cuánto se reevalúa

Máximo **2 páginas** en **docs/model_card.md**. Debe **coincidir con la memoria y con el aviso del demostrador**.

Ejemplo inventado (biblioteca), apartado 2: «Uso fuera de alcance: multar o vetar a una persona concreta.»

CONSULTA · RETO · MEMORIA: ESQUELETO Y REQUISITOS

La memoria técnica: 10 a 15 páginas



Total: 13,5 páginas (el límite es de 10 a 15; los anexos no cuentan)

Se escribe contando lo que ya existe

Cada sección sale de algo hecho: la ficha del Sprint 14, el registro del 15 y la evaluación del 16. **El resumen, al final.**

Cada requisito, en su sección

Los R1 a R10 de la columna azul. R8 también en el README y R9 en el anexo F (tablero y retros). R11, la defensa, es del Sprint 17.